Production Readiness

Blue Harvest · Mortgage Pricing Exceptions

Purpose: identify the exercise trade-offs, explain the changes required before production deployment and set clear verification expectations. The improvements below are proposed work, not completed capabilities.

1. Lightweight Exercise Trade-offs

1.1 Development identity simulation

The local application accepts X-User-Id for a seeded user and reads the role from the database. Anyone who knows a seeded ID can act as that user in Development. The header is rejected outside Development, but no production identity provider is configured. Setting the environment to Production alone does not provide usable production sign-in.

1.2 Single-instance concurrency

A singleton semaphore coordinates workflow reads and writes within one API process. Version checks and an EF concurrency token protect tracked updates, but the semaphore cannot coordinate multiple processes. One request per application and actor/operation/key retry uniqueness are service checks without unique database indexes.

1.3 External application and policy assumptions

Mortgage applications are external GUID references; existence, ownership and eligibility are not checked. Discount limits, read visibility, one request per application and revision rights are exercise policies. Approval expiry, downstream consumption and external revocation are not implemented.

1.4 Local storage and operational setup

Docker Compose runs PostgreSQL with a named volume, local demo credentials and Development settings. Startup automatically applies migrations and seeds users. A volume retains data across restarts but does not provide backups, failover or a recovery guarantee. Compose is a local execution setup rather than a completed production deployment design.

1.5 Audit and observability boundaries

SaveChanges guards protect submitted revisions, decisions, history and retry records from application-level edits or deletion; direct SQL can bypass those guards. Telemetry hooks expose workflow activities and mutation, replay and conflict counters. Production exporters, dashboards, alerting and service targets are not configured.

2. Changes Before Production Deployment

2.1 Confirm the business contract

Business and mortgage-system owners should approve the discount range, required reasons, visibility, independent review rules and whether parallel requests are allowed. Define what happens if terms change after an approved discount has already been used. Agree approval expiry, revocation, retry retention and audit retention before implementing lifecycle changes.

Add a supported integration to validate application existence and eligibility. Define whether downstream consumers pull approved discounts or need a reliable notification. Introduce an outbox or messaging only if a confirmed integration requires delivery guarantees.

2.2 Establish production identity and deployment configuration

Integrate the organization's trusted identity provider, validate token issuer, audience and lifetime, and derive user identity from trusted claims. Map identities to approved business permissions. Disable demo-user builds and seeding in production; retain server-side rejection of demo headers. Restrict API documentation exposure according to the deployment policy.

Use managed secrets and separate environment configuration. Establish HTTPS ingress, secure database connections, network access controls and least-privilege service/database accounts. Review reverse-proxy headers and HTTPS behavior so the API can identify the original request scheme correctly.

2.3 Strengthen persistence and deployment safety

After confirming cardinality rules, add database uniqueness for application requests and scoped retry keys. Handle competing inserts and transaction failures so callers receive the intended conflict or original replay result. Preserve existing data and retry compatibility through a reviewed migration.

Move production schema changes into a controlled release step, with one migration executor, compatibility checks and a recovery plan. Avoid multiple API instances racing to apply migrations. Use versioned deployment artifacts, automated checks and a staged rollout with a defined rollback decision.

2.4 Define release acceptance evidence

- Verify production identity, role restrictions, request ownership and assigned-reviewer checks. Confirm demo impersonation cannot be enabled through deployment configuration.

- Run PostgreSQL concurrency tests against multiple API instances, including duplicate creation, conflicting retry keys, decision-versus-revision races and lost-response replay.

- Demonstrate migration compatibility, backup restoration and recovery from interrupted writes or deployment failures.

- Exercise the complete business flow in a staging environment. Establish performance targets with stakeholders and test against them before approving release.

3. Security, Reliability, Scalability and Monitoring

3.1 Security and privacy

Enforce access rules for the organization's real user population. Review whether all internal users should see all request reasons, and define delegation and approval limits. Apply least privilege to human and service accounts and test cross-user access explicitly.

Classify request reasons, stored response data and audit information. Define encryption, retention and deletion policies that respect audit requirements. Protect credentials, redact sensitive logs and review diagnostic access. Assess browser-session protections, including CSRF if cookies are introduced. Add dependency/image scanning and API rate limits appropriate to expected use.

3.2 Reliability and data recovery

Keep workflow changes, history and the completed retry record in one database transaction. Revalidate that guarantee when changing persistence code. Treat operation scopes, payload hashing and stored response shapes as compatibility contracts: a deployment must not invalidate legitimate retries from an earlier version.

Agree recovery time and acceptable data loss with the business. Configure backups, test restoration and document failover and incident actions. Add liveness and readiness checks with clear dependency behavior; a running process is not sufficient evidence that the service can accept business traffic.

Use explicit timeouts and bounded retries for transient faults. Do not automatically retry business conflicts. If database commit outcome is uncertain, preserve the original idempotency key and input when recovering the operation. Define durable retry retention before introducing expiry or cleanup.

3.3 Scalability and performance

Establish correctness across multiple instances before adding replicas. Replace reliance on process-wide serialization with database-backed coordination and constraints. Review isolation and transaction boundaries for consistent reads where business requirements demand a snapshot.

The current list operation loads all requests, revisions and decisions. Introduce measured query improvements, pagination and suitable indexes when volume requires them. Review connection pools, query plans and resource limits under representative load. Cache only where invalidation preserves the rule that revised terms cannot reuse an older approval.

3.4 Monitoring and incident response

Connect the existing activity source and counters to an approved telemetry collector and backend. Add request latency, error rates, database availability, lock/transaction contention and saturation metrics. Separate expected business conflicts from infrastructure failures.

Define service targets and actionable alerts with named owners. Use correlation/trace IDs to connect errors to logs while protecting sensitive reasons and credentials. Maintain dashboards for service health and a business audit trail for decisions; telemetry is not a replacement for persisted history.

Provide runbooks for failed migrations, database recovery, identity-provider failure, elevated 500 responses and replay anomalies. Test escalation and recovery procedures. Record actual deployment and test evidence against the release commit before declaring production readiness.

3.5 Source Basis and Status

Checked against compose.yaml, frontend Nginx configuration, API startup and development authentication, PricingExceptionService, PricingExceptionsDbContext, GlobalExceptionHandler and PricingExceptionsTelemetry. Backend source paths are under Backend/BlueHarvest.PricingExceptions/src/PricingExceptions.API. Related handoff: Deliverables/README.md and Engineering_Decisions_and_Operational_Guidance.docx.

This is a source-based readiness assessment. No production infrastructure, application behavior or tests were changed or executed as part of creating this document. Local test success alone does not establish production readiness.